

PRÁCTICA · JAVASCRIPT EN EL NAVEGADOR

Un ahorcado, tres caminos

Buenas, soy Ramón Abramo, profesor con más de 25 años de experiencia docente. Hoy os voy a explicar tres formas de programar el mismo juego del ahorcado. Las reglas no cambian; lo que cambia es cómo se organiza el código, desde lo más directo hasta una estructura de profesional.

Solución 1 · Básica lineal | Solución 2 · Teclado y eventos | Solución 3 · Avanzada con clases

MAPA DE LA SESIÓN



¿Qué vamos a hacer hoy?

Este es el recorrido, en este orden.

🧱 Lo común

Las reglas, el dibujo escondido en el SVG, la memoria del juego y las herramientas que comparten las tres.

1 · Básica lineal

Una caja de texto, un botón y variables globales. La lógica del juego, sin adornos.

2 · Teclado y eventos

Teclado fabricado con JavaScript, un solo escuchador para 27 teclas y teclado físico.

3 · Avanzada con clases

Motor, interfaz, sonido y estadísticas, cada uno en su clase.

⚖️ Comparativa

Las tres frente a frente, trampas típicas y preguntas de repaso.

📚 Fuentes

Documentación oficial para ampliar en casa.



Seis hitos que cumplen las tres soluciones

#	HITO	CÓMO SE COMPRUEBA
1	Palabra al azar y tapada	Para `HTML` se ven cuatro huecos
2	Validar la letra	Una letra repetida no quita vidas
3	Destapar aciertos	`A` en `PANTALLA` → `_ A _ _ A _ _ A`
4	Dibujar el muñeco con cada fallo	Cabeza, tronco, brazo izq., brazo der., pierna izq., pierna der.
5	Victoria	Sin huecos → mensaje y juego bloqueado
6	Derrota y revelación	Sexto fallo → ojos en cruz y palabra a la vista

Se construye por hitos, uno detrás de otro, probando cada uno antes de pasar al siguiente. Si intentáis escribirlo todo de golpe, cuando falle no sabréis dónde.



El muñeco ya está dibujado… y escondido

```
<svg id="svg-ahorcado" viewBox="0 0 200 240">
  <line ... class="trazo-horca" />
  <circle id="parte-cabeza" class="parte-oculta" />
  <line id="parte-cuerpo" class="parte-oculta" />
  <line id="parte-brazo-izq" class="parte-oculta" />
  <line id="parte-brazo-der" class="parte-oculta" />
  <line id="parte-pierna-izq" class="parte-oculta" />
  <line id="parte-pierna-der" class="parte-oculta" />
  <g id="parte-cara-derrota" class="parte-oculta">...</g>
</svg>
```

Ninguna de las tres soluciones dibuja nada. Todas las piezas están en el HTML desde el principio, con la clase `parte-oculta`. Para que aparezca una, basta con añadirle la clase `visible`[1].

```
partesAhorcado[fallos - 1].classList.add('visible');
```



🔍 **Para entenderlo:** un teatro de sombras con todas las figuras detrás de la cortina. Con cada fallo encendéis un foco más.

El «menos uno» sale porque el primer fallo es el 1 y la primera posición del array es la 0.

LO COMÚN · LA MEMORIA



¿Qué tiene que recordar el juego?

Haced la prueba mental: estáis jugando con papel y alguien ha dicho ya la A, la L y la E. ¿Qué tenéis apuntado?

DATO	EJEMPLO
Palabra secreta	`'PANTALLA'`
Tablero	`['_','A','_','_','A','L','L','A']`
Letras probadas	`A, L, E`
Fallos	`1`
¿Ha terminado?	`false`

Las tres soluciones guardan exactamente esto. Cambia el envoltorio:

- **1:** variables globales sueltas.
- **2:** variables globales, pero la palabra es un objeto con categoría y pista, y las letras probadas van en un `Set` [2].
- **3:** todo dentro de un objeto de la clase `AhorcadoEngine` [3].



🔍 **Para entenderlo:** el tablero es un calendario de adviento: todas las ventanitas cerradas al principio, y cada acierto abre las que llevan esa letra.



Cuatro piezas de JavaScript que salen en las tres

Elegir al azar

```
const i = Math.floor(Math.random() * lista.length);
```

`Math.random()` da un decimal entre 0 y 1, sin llegar a 1[4]. Multiplicado por la longitud y redondeado hacia abajo con `Math.floor()` [5], da una posición válida.

¿Está dentro?

```
'PANTALLA'.includes('A');  // true
```

`includes()` funciona con textos[6] y con arrays[7].

El bucle que destapa

```
for (let i = 0; i < palabra.length; i++) {
  if (palabra[i] === letra) tablero[i] = letra;
}
```

Recorre la palabra entera, así destapa todas las apariciones y no solo la primera.

Escuchar al jugador

```
boton.addEventListener('click', jugar);
```

«Cuando pase esto, llama a esta función»[8]. Lo que cambia entre soluciones es *dónde* se pone.



1

SOLUCIÓN 1

Básica lineal

Una caja de texto, un botón «Probar» y las reglas escritas de arriba abajo, tal y como las diríais en voz alta. La mejor para entender el juego antes de complicarlo.

SOLUCIÓN 1 · HITO 1



Sacar una palabra y taparla

```
function iniciarJuego() {
  const indice = Math.floor(Math.random() * PALABRAS.length);
  palabraSecreta = PALABRAS[indice];

  letrasAdivinadas = [];
  for (let i = 0; i < palabraSecreta.length; i++) {
    letrasAdivinadas.push('_');
  }

  letrasUsadas = [];
  fallos = 0;
  juegoTerminado = false;
  // ... esconder piezas, activar la caja y pintar
}
```

Tres bloques, tres ideas: sortear, tapar y poner el marcador a cero.

El tablero se pinta con `letrasAdivinadas.join(' ')`[9], que convierte `['_', '_', '_']` en el texto `_ _ _`.



🔍 **Para entenderlo:** sacáis una papeleta de la bolsa y dibujáis en la pizarra tantas rayitas como letras tiene.

`iniciarJuego()` se usa al cargar y cada vez que se pulsa «Nueva Palabra». Por eso pone todo a cero: la segunda partida no puede heredar nada de la primera.

SOLUCIÓN 1 · HITO 2



El portero de la discoteca

```
function procesarLetra() {
  if (juegoTerminado) return;

  const letra = inputLetra.value.trim().toUpperCase();
  inputLetra.value = '';

  if (!/^[A-ZÑ]$/.test(letra)) {
    mostrarMensaje('Introduce una letra válida (A-Z).', 'aviso');
    return;
  }
  if (letrasUsadas.includes(letra)) {
    mostrarMensaje(`La letra "${letra}" ya está probada.`, 'aviso');
    return;
  }
  letrasUsadas.push(letra);
  // ... acierto o fallo
}
```

Antes de jugar la letra, se limpia: `trim()` quita espacios[10] y `toUpperCase()` la pasa a mayúscula[11], porque las palabras del banco están en mayúsculas.

Después, dos filtros. La expresión `/^[A-ZÑ]$/` con `test()`[12] solo deja pasar **una** letra de la A a la Z o la Ñ. Y `includes` descarta las repetidas.



🔍 **Para entenderlo:** un portero que mira primero el DNI y luego la lista de invitados. Si falla cualquiera de las dos cosas, `return`, a la calle, y el juego se queda como estaba.

SOLUCIÓN 1 · HITOS 3 Y 4



¿Está o no está?

```
if (palabraSecreta.includes(letra)) {
  for (let i = 0; i < palabraSecreta.length; i++) {
    if (palabraSecreta[i] === letra) {
      letrasAdivinadas[i] = letra;
    }
  }
  mostrarMensaje(`¡Acierto! ...`, 'exito');
} else {
  fallos++;
  dibujarSiguienteParte(fallos);
  mostrarMensaje(`¡Fallo! ...`, 'error');
}

actualizarPantalla();
verificarFinJuego();
```

Un `if / else` y nada más. Si está, el bucle recorre la palabra entera y destapa cada posición que coincide. Si no está, se suma un fallo y se enciende una pieza.

Las dos últimas líneas se ejecutan siempre: repintar la pantalla y preguntar al árbitro si el partido ha terminado.



⚠️ **Ojo:** un `break` dentro del bucle, o usar `indexOf`, destapa solo la primera A de `PANTALLA`. Es el fallo más típico de este hito.

SOLUCIÓN 1 · TRAZA EN LA PIZARRA



Seguimos a mano una partida con `PANTALLA`

ESCRIBE	FILTRO	¿ESTÁ?	TABLERO	PROBADAS	FALLOS
`a`	pasa como `A`	sí	`_ A _ _ A _ _ A`	A	0
`e`	pasa como `E`	no	`_ A _ _ A _ _ A`	A, E	1 → cabeza
`a`	repetida → `return`	—	sin cambios	A, E	1
`7`	no es letra → `return`	—	sin cambios	A, E	1
`l`	pasa como `L`	sí	`_ A _ _ A L L A`	A, E, L	1
`p`, `n`, `t`	pasan	sí	`P A N T A L L A`	A, E, L, P, N, T	1 → ¡victoria!

Fijaos en las filas 3 y 4: ni el tablero ni los fallos cambian. Eso es exactamente el hito 2.

SOLUCIÓN 1 · BALANCE



Lo bueno y lo mejorable

👍 Lo que hace bien

- Se lee de arriba abajo, como la regla del juego.
- Pocas herramientas: arrays, `for`, `if` y `getElementById` [13].
- Ideal para aprender a depurar: cada variable se ve en la consola.

👎 Lo mejorable

- Escribir en una caja y pulsar «Probar» es incómodo para jugar.
- Todo son variables globales: cualquier función puede cambiar cualquier cosa.
- El jugador no ve qué letras le quedan por probar.

La Solución 2 ataca justo la primera y la tercera queja: un teclado en pantalla que se va apagando.



2

SOLUCIÓN 2

Teclado y eventos

Las reglas son las mismas. Cambia la forma de jugar: un teclado de 27 botones que fabrica el propio JavaScript, un único escuchador para todos y el teclado del ordenador funcionando a la vez.

SOLUCIÓN 2 · LOS DATOS



Palabras con ficha y un `Set` para las usadas

```
const BANCO_PALABRAS = [
  { palabra: 'CSS', categoria: 'Diseño Web',
    pista: 'Hojas de estilo en cascada...' },
  // ...
];

let partidaActual = null;
let letrasAdivinadas = [];      // ['', 'S', 'S']
let letrasUsadas = new Set();
```

Cada palabra viaja con su categoría y su pista. Al sortear, se guarda la ficha entera en `partidaActual` y se usa `partidaActual.palabra`, `partidaActual.pista`…

Las letras probadas van en un `Set` [2]: una colección que no admite repetidos. Se trabaja con `add`, `has` y `clear`.

Y el tablero se rellena con `Array(n).fill('')` [14]: huecos vacíos en vez de guiones, porque cada letra se pinta en su propia casilla.



⚠️ **Ojo:** por eso la victoria se comprueba con `!letrasAdivinadas.includes('')`, no con `'_'`. Quien copia de la Solución 1 gana nada más empezar.



27 botones que no están en el HTML

```
function renderizarTecladoVirtual() {
  tecladoVirtual.innerHTML = '';
  LETRAS_ALFABETO.forEach(letra => {
    const btn = document.createElement('button');
    btn.type = 'button';
    btn.className = 'tecla';
    btn.textContent = letra;
    btn.dataset.letra = letra;
    tecladoVirtual.appendChild(btn);
  });
}
```

En el `index.html` el teclado es un `div` vacío. Los botones los crea JavaScript con `createElement`[15] y los mete dentro con `appendChild`[16].

Cada botón lleva su letra apuntada en `dataset.letra`[17], que en el HTML aparece como `data-letra="A"`.



🔍 **Para entenderlo:** la etiqueta con el nombre en la taquilla del gimnasio. Lo de dentro da igual: la etiqueta dice de quién es.

¿La ventaja? Si mañana queréis añadir la Ç, se cambia una cadena de texto, no el HTML.

SOLUCIÓN 2 · DELEGACIÓN DE EVENTOS



Un solo conserje para 27 teclas

```
tecladoVirtual.addEventListener('click', evento => {
  const tecla = evento.target.closest('.tecla');
  if (tecla && !tecla.disabled) {
    jugarLetra(tecla.dataset.letra);
  }
});
```

Cuando hacéis clic en una tecla, el evento no se queda en ella: sube por sus padres. Es el burbujeo[18]. Así que basta un escuchador en el contenedor.

`evento.target`[19] dice dónde se hizo clic; `closest('.tecla')`[20] sube hasta la tecla, o da `null` si se pulsó en un hueco.

```
<body>
        ↑
<section class="panel-juego">
        ↑
<div id="teclado-virtual"> ← aquí escucha el conserje
        ↑ el clic sube
<button class="tecla" data-letra="L"> ← clic aquí
```



🔍 **Para entenderlo:** el conserje de un edificio. En vez de un portero por piso, uno solo en la entrada pregunta «¿a qué piso va?».

SOLUCIÓN 2 · LO QUE PIENSA EL CONSERJE



Seguimos a mano un clic en la tecla `L`

En clase lo veis funcionando con un teclado de verdad. Aquí tenéis lo que «piensa» el escuchador del contenedor cuando la palabra es `BUCLE` y pulsáis la `L`:

PASO	EXPRESIÓN	VALOR
1	`evento.target`	el botón `<button class="tecla" data-letra="L">`
2	`evento.target.closest('.tecla')`	el mismo botón (ya es una tecla)
3	`tecla.dataset.letra`	`'L'`
4	`letrasUsadas.has('L')`	`false` → se juega
5	`'BUCLE'.includes('L')`	`true` → tecla verde
6	`letrasAdivinadas`	`['', '', '', 'L', '']`
7	`letrasUsadas`	`Set {'L'}`

Si en vez de en una tecla pulsáis en el hueco entre dos, `evento.target` es el propio contenedor, `closest('.tecla')` devuelve `null` y el `if` no deja pasar nada.

SOLUCIÓN 2 · TECLADO FÍSICO



Dos puertas, un mismo pasillo

```
window.addEventListener('keydown', evento => {
  if (evento.ctrlKey || evento.altKey || evento.metaKey) return;
  const letra = evento.key.toUpperCase();
  if (LETRAS_ALFABETO.includes(letra)) {
    jugarLetra(letra);
  }
});
```

El evento `keydown` [21] en `window` se entera de cualquier tecla pulsada en la página. `evento.key` [22] trae el carácter.

Lo bonito es la última línea: llama a la **misma** `jugarLetra()` que el ratón. Toda la lógica está en un solo sitio; las entradas son solo puertas.



⚠️ **Ojo:** sin la primera línea, `Ctrl` + `R` para recargar contaría como haber pulsado la R.



Lo bueno y lo mejorable

👍 Lo que hace bien

- Mucho más cómoda de jugar: ratón, dedo o teclado.
- Un solo escuchador para 27 teclas, aunque se regeneren en cada partida.
- Toda la lógica de una jugada en una función: `jugarLetra()`.

👎 Lo mejorable

- `jugarLetra()` decide las reglas *y* pinta la pantalla. Mezcla dos trabajos.
- Siguen siendo variables globales.
- Al recargar, se pierde todo: no hay historial de partidas.



3

SOLUCIÓN 3

Avanzada con clases

Cada responsabilidad en su clase: un motor que sabe las reglas y no toca la pantalla, una interfaz que pinta y no decide, y dos ayudantes para el sonido y las estadísticas. Con niveles de dificultad de propina.

SOLUCIÓN 3 · EL REPARTO



Cuatro clases, cuatro trabajos

CLASE	SU TRABAJO	LO QUE NO HACE NUNCA
`AhorcadoEngine`	Sortear, comprobar letras, contar fallos, decidir victoria o derrota.	Tocar el HTML.
`AhorcadoUI`	Pintar teclado, casillas, salud y ventanas; escuchar al jugador.	Decidir si una letra es acierto.
`SoundSynthesizer`	Generar pitidos con la Web Audio API[23], sin archivos de sonido.	Saber nada del juego.
`StatsRepository`	Guardar partidas y rachas en `localStorage`[24].	Saber nada de la pantalla.



🔍 **Para entenderlo:** un partido de fútbol. El árbitro (motor) aplica el reglamento; el marcador del estadio (interfaz) enseña el resultado; la megafonía (sonido) pita; y la federación (estadísticas) apunta los resultados en su libro.

SOLUCIÓN 3 · EL MOTOR



El árbitro devuelve un informe, no pinta nada

```
probarLetra(letra) {
  if (this.juegoTerminado || this.letrasUsadas.has(letra)) return null;

  this.letrasUsadas.add(letra);
  const acierto = this.itemActual.palabra.includes(letra);

  if (acierto) { /* rellenar this.letrasAdivinadas */ }
  else { this.fallos++; }

  const max = this.configDificultades[this.dificultad].maxFallos;
  const victoria = !this.letrasAdivinadas.includes('');
  const derrota = this.fallos >= max;
  if (victoria || derrota) this.juegoTerminado = true;

  return { letra, acierto, fallos: this.fallos, maxFallos: max,
           victoria, derrota, palabraSecreta: this.itemActual.palabra };
}
```

Las reglas son las de siempre, pero todo lo «del objeto» lleva `this.` delante[25]: `this.fallos`, `this.letrasUsadas`…

Y en vez de pintar, **devuelve un objeto** con el resultado de la jugada. Quien lo llame decidirá qué hacer con él.

Las vidas dependen de la dificultad: 8 en fácil, 6 en normal, 5 en difícil. Y en difícil hay además un reloj de 45 segundos: la interfaz lo descuenta con `setInterval`[26] y, al llegar a cero, el motor da la partida por perdida con `tiempoAgotado()`.



✅ **Probadlo:** el motor funciona sin pantalla. En la consola: `const m = new AhorcadoEngine(); m.iniciarNuevaPartida(); m.probarLetra('A');`



El marcador lee el informe y pinta

```
jugar(letra) {
  const res = this.engine.probarLetra(letra);
  if (!res) return;

  const tecla = this.dom.teclado.querySelector(`[data-letra="${letra}"]`);
  tecla.disabled = true;
  tecla.classList.add(res.acierto ? 'acierto' : 'fallo');

  if (res.acierto) {
    this.audio.playSuccess();
    this.renderPalabra();
  } else {
    this.audio.playError();
    this.dom.partesSvg[res.fallos - 1].classList.add('visible');
    this.actualizarSalud();
  }
  if (res.victoria || res.derrota) this.finPartida(res);
}
```

Fijaos en lo que **no** hay: ni un `includes`, ni un bucle sobre la palabra. La interfaz pregunta al motor y obedece.

Los escuchadores se escriben con funciones flecha[27]:

```
this.dom.teclado.addEventListener('click', evento => {
  const tecla = evento.target.closest('.tecla');
  if (tecla) this.jugar(tecla.dataset.letra);
});
```



⚠️ **Ojo:** con `function (evento) {...}`, dentro del escuchador `this` sería el teclado, no la interfaz, y `this.jugar` no existiría.



Memoria entre visitas y sonido sin archivos

📒 `StatsRepository`

```
save() {
  localStorage.setItem(this.STORAGE_KEY,
    JSON.stringify(this.stats));
}
```

`localStorage` solo guarda texto[24]. Por eso el objeto de estadísticas se convierte con `JSON.stringify` al guardar y con `JSON.parse` al leer[28].



🔍 **Para entenderlo:** una libreta que el navegador deja en un cajón. Cerráis la página y la libreta sigue ahí.

🔊 `SoundSynthesizer`

```
const osc = this.audioCtx.createOscillator();
osc.frequency.setValueAtTime(587.33, ...);
osc.start();
```

La Web Audio API[23] fabrica el sonido con un oscilador: una frecuencia, una forma de onda y una duración. No hay ni un MP3 en la carpeta.

Y el botón de silencio también se recuerda en `localStorage`.



Lo bueno y lo mejorable

👍 Lo que hace bien

- Cada clase se entiende y se prueba por separado.
- El motor podría usarse en otro diseño de pantalla sin tocar una línea.
- Estadísticas que sobreviven a la recarga y niveles de dificultad.

👎 Lo mejorable

- Mucho más código para el mismo juego: hay que conocer clases, `this` y flechas.
- Para un ejercicio pequeño puede ser matar moscas a cañonazos.
- Más piezas, más sitios donde equivocarse con un nombre.

COMPARATIVA · FRENTE A FRENTE



Las tres soluciones en una tabla

CRITERIO	1 · BÁSICA	2 · TECLADO	3 · CLASES
Cómo entra la letra	Caja de texto + botón o `Enter`	Teclado en pantalla + teclado físico	Igual que la 2
Escuchadores de letras	2 (botón y caja)	1 en el contenedor + 1 en `window`	Igual que la 2, dentro de la clase
Banco de palabras	Array de textos	Array de objetos con categoría y pista	Igual que la 2
Letras probadas	Array	`Set`	`Set` dentro del motor
Dónde están las reglas	En `procesarLetra()`	En `jugarLetra()`, mezcladas con la pantalla	Solo en `AhorcadoEngine`
Vidas	6	6	8, 6 o 5 según dificultad, y 45 s de reloj en difícil
Extras	—	Pista y corazones	Sonido, estadísticas, ventana de fin
Para quién	Iniciación	Práctica estándar de clase	Reto y portafolio

COMPARATIVA · EL MISMO MOMENTO, TRES CÓDIGOS



¿Cómo se entera cada una de que habéis jugado una letra?

Solución 1

```
btnProbar.addEventListener(
  'click', procesarLetra);
inputLetra.addEventListener(
  'keydown', e => {
    if (e.key === 'Enter')
      procesarLetra();
  });
```

La letra se lee de la caja dentro de `procesarLetra()`.

Solución 2

```
tecladoVirtual.addEventListener(
  'click', e => {
    const t = e.target
      .closest('.tecla');
    if (t) jugarLetra(
      t.dataset.letra);
  });
```

La letra llega como parámetro, venga del ratón o del teclado.

Solución 3

```
this.dom.teclado
  .addEventListener(
  'click', e => {
    const t = e.target

.closest('.tecla');
    if (t) this.jugar(
      t.dataset.letra);
  });
```

Igual que la 2, pero llama a un método y este pregunta al motor.

COMPARATIVA · TRAMPAS TÍPICAS



Los fallos que más veo en esta práctica

SÍNTOMA EN PANTALLA	CAUSA	ARREGLO
Solo se destapa la primera A	`indexOf` o un `break` en el bucle	Recorrer la palabra entera con `for`
El primer fallo dibuja el tronco	Índice sin el `- 1`, o `fallos++` después de dibujar	`partes[fallos - 1]` tras sumar el fallo
Una letra repetida quita vida	Falta el `return` tras el aviso	Salir de la función en cuanto falla un filtro
«Victoria» nada más empezar (Solución 2 y 3)	Se busca `'_'` en vez de `''`	`!letrasAdivinadas.includes('')`
La segunda partida empieza con piezas	`iniciarJuego()` no quita la clase `visible`	Recorrer las piezas con `classList.remove`
`this.jugar is not a function`	Escuchador escrito con `function`	Función flecha

COMPROBAD LO APRENDIDO



Cinco preguntas rápidas

1. ¿Qué cinco cosas tiene que recordar el juego durante una partida?
2. ¿Por qué el primer fallo enciende `partesAhorcado[0]` y no `partesAhorcado[1]`?
3. En la Solución 2, ¿por qué basta un escuchador para las 27 teclas aunque se creen de nuevo en cada partida?
4. ¿Qué devuelve `probarLetra()` en la Solución 3 y por qué no pinta nada en pantalla?
5. ¿Por qué hay que usar `JSON.stringify` antes de guardar las estadísticas en `localStorage`?

Si alguna os cuesta, volved a su bloque. Las respuestas están todas en esta presentación.



Fuentes y Documentación Oficial

1. MDN Web Docs — Element.classList
2. MDN Web Docs — Set
3. MDN Web Docs — Clases
4. MDN Web Docs — Math.random()
5. MDN Web Docs — Math.floor()
6. MDN Web Docs — String.prototype.includes()
7. MDN Web Docs — Array.prototype.includes()
8. MDN Web Docs — EventTarget.addEventListener()
9. MDN Web Docs — Array.prototype.join()
10. MDN Web Docs — String.prototype.trim()
11. MDN Web Docs — String.prototype.toUpperCase()
12. MDN Web Docs — RegExp.prototype.test()
13. MDN Web Docs — Document.getElementById()
14. MDN Web Docs — Array.prototype.fill()
15. MDN Web Docs — Document.createElement()
16. MDN Web Docs — Node.appendChild()
17. MDN Web Docs — HTMLElement: dataset property
18. MDN Web Docs — Burbujeo de eventos
19. MDN Web Docs — Event.target
20. MDN Web Docs — Element.closest()
21. MDN Web Docs — Element: keydown event
22. MDN Web Docs — KeyboardEvent: key property
23. MDN Web Docs — Web Audio API
24. MDN Web Docs — Window.localStorage
25. MDN Web Docs — this
26. MDN Web Docs — setInterval()
27. MDN Web Docs — Funciones flecha
28. MDN Web Docs — JSON